

DevOps Master Prep Guide

Complete 15-Day Plan + Knowledge Reference

Built for Sameer Khan | Senior DevOps / Cloud Engineer

Daily Time	10:00 AM – 1:00 PM (3 hours)
Total Duration	15 days = 45 hours
Goal	Interview-ready, confident, rust removed
Approach	70% hands-on, 30% theory + deep concepts
Coverage	AWS, K8s, Terraform, CI/CD, Observability, System Design, Behavioral
Total Pages	~50 pages of curated content

Bhai, is document mein sab kuch hai:

- 15-day daily plan (kya padhna, kahan se padhna, kitna time)
- Har topic ka deep concept explanation
- 100+ verified resource links (YouTube, docs, blogs, labs)
- Free hands-on environments setup guide
- 50+ interview questions with answer hints
- System design templates
- Behavioral story framework

Ye tera 'go-to' document hai. 15 din rozana isi se padhega.

Table of Contents

Section	Page Topic
1.	How to Use This Guide + Daily Routine
2.	Hands-on Environment Setup (Free Resources)
3.	15-Day Roadmap Overview
4.	PHASE 1 (Day 1-3): Foundation
5.	PHASE 2 (Day 4-7): Cloud Core (AWS + Azure + GCP)
6.	PHASE 3 (Day 8-10): Kubernetes Deep
7.	PHASE 4 (Day 11-12): IaC + CI/CD
8.	PHASE 5 (Day 13): Observability + Debugging
9.	PHASE 6 (Day 14-15): Mock Interviews + System Design
10.	Deep Concept Reference (Quick Lookup)
11.	Master Resource List (All Links)
12.	Top 50 Interview Questions
13.	System Design Templates
14.	Behavioral Story Framework (STAR)
15.	Final Cheatsheet (Day-of-Interview)

1. How to Use This Guide

Daily Routine (3 hours = 10 AM to 1 PM)

Time	Activity	Why
10:00 – 10:15	Previous day quick recap	Memory consolidation
10:15 – 11:30	New topic — video + docs reading	Core learning (75 min)
11:30 – 11:45	Break (chai, walk, no phone)	Brain reset
11:45 – 12:45	Hands-on lab / practice	Implementation muscle
12:45 – 1:00	Notes likho apne words mein	Retention boost

Golden Rules

- **Active learning over passive:** Video dekhne ke baad apne words mein likh — agar likh nahi paya toh samjha nahi.
- **Hands-on mandatory:** Sirf padhne se interview crack nahi hota. Roz kuch deploy/run kar.
- **Loud thinking:** Practice questions ka jawab loud bol ke de, jaise interview mein. Phone pe record kar, sun.
- **Resume-first:** Har topic padhte waqt soch — 'mere resume ke kis project mein use hua tha?' Wahan jodke yaad rakh.
- **Don't chase perfection:** 80% coverage of 100 topics > 100% of 20 topics. Move on.
- **Sleep 7+ hours:** Brain rest ke bina memory consolidate nahi hoti. Deadline ke chakkar mein neend mat khona.

2. Hands-on Environment Setup

Tu likh raha tha 'jo implement kiya uska access nahi hai.' Ye ek bahut common problem hai — company ke production access chala jaata hai job switch pe. Solution ye hai: **apna personal lab setup kar**. Free aur paid options dono hain — focus free pe rakh.

Free / Cheap Options (Priority Order)

Tool	Use Case	Cost	Setup
Minikube	Local K8s practice — daily use	FREE	minikube.sigs.k8s.io
Kind	Multi-node K8s on laptop	FREE	kind.sigs.k8s.io
KillerCoda	Browser K8s/Docker labs (1hr free)	FREE	killercoda.com
Play with K8s	Free 4-hour K8s session	FREE	labs.play-with-k8s.com
Play with Docker	Browser Docker labs	FREE	labs.play-with-docker.com
LocalStack	AWS services locally simulate	FREE	localstack.cloud
GitHub Codespaces	Cloud dev env (60 hrs/month free)	FREE	github.com/codespaces
AWS Free Tier	Real AWS — 12 months free	FREE/Cheap	aws.amazon.com/free
GCP Free Tier	\$300 credits, 90 days	FREE	cloud.google.com/free
Azure Free	\$200 credits, 30 days	FREE	azure.microsoft.com/free
KodeKloud Engineer	Real-world tasks (free tier)	FREE	kodekloud.com/engineer

Day 1 Setup Checklist (Mandatory)

- **Minikube install:** Download kar, minikube start chala, kubectl get nodes verify.
- **kubectl + helm install:** Latest version. kubectl version, helm version.
- **Docker Desktop:** Tu use karta hi hai, just verify chal raha hai.
- **AWS CLI configured:** aws configure, free tier account ka access key.
- **Terraform installed:** terraform version.
- **VS Code:** Extensions install — Kubernetes, Terraform, Docker, YAML, GitLens.
- **KillerCoda bookmark:** Account banao, ek scenario try karo.
- **GitHub repo banao:** 'devops-prep-2026' — saari practice yahan commit karega.

Tip — Cost protection: AWS account mein **Billing Alert** set karo — \$5 cross hote hi email aaye. Free tier mein bhi accidentally bill aa sakta hai (NAT Gateway, EKS control plane \$0.10/hour charge karta hai). EKS use karke turant destroy karo.

3. 15-Day Roadmap Overview

Phase	Days	Focus Areas	Outcome
Phase 1: Foundation	1-3	Linux, Networking, Resume Stories	Behavioral confidence
Phase 2: Cloud Core	4-7	AWS deep, Azure refresh, GCP touch	Cloud rust removed
Phase 3: Kubernetes	8-10	K8s deep, Helm, troubleshooting	K8s production-ready
Phase 4: IaC + CI/CD	11-12	Terraform, Pulumi, GitHub Actions	Pipeline fluency
Phase 5: Observability	13	Prometheus, Grafana, OTEL, debug	Triage skills
Phase 6: Polish	14-15	Mock interviews, system design	Interview-ready

PHASE 1: Foundation (Day 1-3)

Goal: Linux/networking ki rust hatao, aur sabse important — apne resume ke har project ki STAR-format kahani ready karo. Behavioral round ka 80% yahan se cover hoga.

Day 1: Setup + Resume Story Bank Banao

Topics + Deep Concept Notes

- **STAR Format Story Building (har resume project ke liye)**

STAR = Situation, Task, Action, Result. Har project ko 4 paragraph mein structure kar. Example: *Vault to SSM migration* — **Situation:** 50+ services HashiCorp Vault use kar rahe the, license cost zyada. **Task:** AWS-native solution pe migrate karna without downtime. **Action:** ESO setup, parallel sync, gradual cutover, rollback plan. **Result:** \$X/month savings, zero downtime, 50+ services migrated. Numbers add karna mandatory.

- **Common Behavioral Questions Pre-prep**

Top 10 hain: (1) Apna sabse bada production failure, (2) Team conflict, (3) Tight deadline, (4) Cross-team disagreement, (5) Mentoring junior, (6) Pushed back on bad decision, (7) Learned new tech quickly, (8) Multiple priorities balance, (9) Why leaving current job, (10) Where do you see yourself in 5 years.

- **Hands-on Environment Setup**

Day 1 mein hi setup karna mandatory hai — agle 14 din lab ready chahiye. Minikube, kubectl, AWS CLI, Terraform, VS Code extensions sab install karo aaj.

Resources (in order)

- **STAR template:** <https://www.themuse.com/advice/star-interview-method> (free article)
- **Behavioral Q&A:** <https://github.com/viraptor/reverse-interview> (DevOps focused)
- **YouTube:** 'Top DevOps Behavioral Questions' by TechWorld with Nana — <https://www.youtube.com/c/TechWorldwithNana>
- **Minikube install:** <https://minikube.sigs.k8s.io/docs/start/>
- **AWS CLI:** <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

Hands-on Practice

- Minikube install + start, deploy ek nginx pod, expose karo as service.
- AWS Free Tier login check karo, billing alert set karo \$5 pe.
- GitHub repo banao 'devops-prep-2026', README mein 15-day plan paste karo.
- Apne resume ke 6 projects ki STAR stories likh — Notion ya Google Doc mein.

End-of-day Deliverable

Document jisme tere har project (TokenTide, Compliance Innovation ke 3 projects, Cybage ke 4 projects) ka 200-word STAR story ho. Numbers + impact zaroor.

Day 2: Linux Fundamentals Refresh

Topics + Deep Concept Notes

- **File System + Permissions**

chmod, chown, umask deep: Numeric (755) vs symbolic (u+x) notation. SUID, SGID, sticky bit. **Yaad rakh:** /etc/passwd, /etc/shadow, /etc/group, /var/log/, /proc/, /sys/. Pod containers Linux pe hi chalte hain — ye samajh ke K8s clear hota hai.

- **Process Management**

ps aux | grep, top vs htop, kill -9 vs kill -15 (SIGKILL vs SIGTERM — ye interview favorite hai).

SIGTERM = graceful shutdown (app cleanup karta hai), **SIGKILL = immediate** (no cleanup). K8s pods ko terminate karta hai pehle SIGTERM, phir grace period ke baad SIGKILL.

- **Networking Commands (DevOps daily use)**

ss -tunlp (modern netstat), **lsof -i :8080** (port kis process pe hai), **dig +trace** (DNS recursive resolution), **curl -v** (verbose HTTP), **tcpdump -i any port 443** (packet capture). Ye 5 commands se 80% network debugging ho jaati hai.

- **systemd Basics**

systemctl start/stop/status/enable/restart [service]. **journalctl -u kubelet -f** (K8s node debug ke liye gold). systemd unit files: /etc/systemd/system/. Service crash kyun hua — yahan se patta lagega.

- **Shell Scripting Refresh**

set -euo pipefail (production scripts ka standard — error pe fail, undefined vars pe fail, pipe error pe fail). Functions, arrays, here-docs, traps. Ek production-grade backup script likh sakna chahiye.

Resources (in order)

- **YouTube:** 'Linux Crash Course' by Learn Linux TV — <https://www.youtube.com/c/LearnLinuxTV>
- **Cheatsheet:** <https://github.com/trinib/Linux-Bash-Commands>
- **Interactive:** <https://linuxjourney.com> (free, networking section 30 min)
- **Shell scripting:** <https://www.shellscript.sh> + <https://learnxinyminutes.com/docs/bash/>
- **SIGTERM vs SIGKILL:** <https://major.io/p/sigterm-linux/> (short article)

Hands-on Practice

- Minikube pod ke andar kubectl exec -it kar, sab Linux commands wahan run kar.
- Ek shell script likh: AWS S3 buckets list, har ek ka size, JSON output, error handling.
- journalctl -u kubelet --since '10 min ago' chala Minikube node pe.
- Ek pod intentionally OOM kar (memory limit 50Mi, stress tool run kar), kubectl describe se SIGKILL dekh.

End-of-day Deliverable

20 commands ki personal cheatsheet (one-liner explanation) jo interview mein 'troubleshooting' question pe aaye to use kar sake.

Day 3: Networking Deep — DevOps Lens

Topics + Deep Concept Notes

- **OSI Model — Practical Layers**

Layer 3 (Network): IP, routing — VPC, subnets, security groups yahan kaam karte hain. **Layer 4 (Transport):** TCP/UDP — NLB, security groups port-level filtering. **Layer 7 (Application):** HTTP, HTTPS — ALB, Ingress controllers, Cloudflare WAF. **Yaad rakh:** Layer batao to interviewer samajh jaata hai tu network samajhta hai.

- **TCP vs UDP Deep**

TCP: Connection-oriented, 3-way handshake (SYN, SYN-ACK, ACK), reliable, ordered, slow. **UDP:** Connectionless, fast, unreliable, no order. **Use cases:** TCP = HTTP, SSH, DB connections. UDP = DNS, video streaming, gaming. **Common ports:** 22 SSH, 80 HTTP, 443 HTTPS, 53 DNS, 3306 MySQL, 5432 Postgres, 6379 Redis.

• DNS Deep Dive

Records: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), TXT (verification, SPF), NS (nameserver). **TTL:** Cache duration — short TTL (60s) for frequent changes, long TTL (24h) for stable records. **Resolution flow:** Browser → OS cache → resolver (ISP) → root → TLD (.com) → authoritative → answer. **Cloudflare quirk:** 'Proxied' (orange cloud) vs 'DNS-only' (grey) — proxied means CF is in path, can DDoS protect + hide origin IP.

• HTTP/HTTPS + TLS

Status codes: 2xx success, 3xx redirect, 4xx client error, 5xx server error. **Common:** 200 OK, 301 (perm redirect), 401 (auth fail), 403 (forbidden), 404, 429 (rate limit), 500, 502 (bad gateway), 503 (unavailable), 504 (gateway timeout). **TLS handshake:** ClientHello → ServerHello + cert → key exchange → encrypted comm. **HTTPS = HTTP over TLS.** Cert chain: Root CA → Intermediate → Domain cert.

• Load Balancers — L4 vs L7

L4 (NLB, AWS NLB): TCP/UDP level, fast, no inspection of content, can do HTTPS pass-through. **L7 (ALB, Cloudflare, Nginx):** HTTP-aware, path-based routing, header manipulation, WAF, slower but smarter. **Sticky sessions:** Same client → same backend (cookie or IP based). Stateful apps mein zaroori.

• Subnetting + CIDR

/24 = 256 IPs (254 usable — 2 reserved for network + broadcast). **/16 = 65536 IPs.** /20 = 4096. /28 = 16. **Formula:** $2^{(32-\text{prefix})}$. AWS VPC mein /16, subnets /20 ya /24 banate hain. Interview mein zaroor poochhte hain — **/24 mein kitne hosts? = 254.**

Resources (in order)

- **YouTube:** 'Computer Networking Course' by freeCodeCamp — <https://www.youtube.com/watch?v=qiQR5rTSshw> (5 hour, watch 2x)
- **Article:** RFC 1180 TCP/IP Tutorial — <https://datatracker.ietf.org/doc/html/rfc1180>
- **Interactive subnetting:** <https://cidr.xyz>
- **DNS visual guide:** <https://howdns.works>
- **HTTPS visual:** <https://howhttps.works>
- **Cloudflare Learning Center:** <https://www.cloudflare.com/learning/> (gold standard)

Hands-on Practice

- dig +trace google.com chala, root → TLD → authoritative dekho.
- curl -v https://api.github.com — full TLS handshake + headers analyze kar.
- tcpdump -i any port 80 -A chala, ek HTTP request capture kar (Minikube ya local).
- Cloudflare free account banao (agar nahi hai), ek domain ke saath proxy mode try kar.
- Subnet calculator se 5 problems solve kar — given /22, kitne subnets of /26 ban sakte?

End-of-day Deliverable

5 questions ke answers ready: (1) DNS resolution flow, (2) TCP vs UDP, (3) L4 vs L7 LB, (4) HTTPS handshake, (5) /24 mein kitne usable IPs.

PHASE 2: Cloud Core (Day 4-7)

Goal: AWS deep coverage, Azure refresh (AZ-204 hai), GCP basics. Tera 'cloud weak' wala dukhda yahan se khatam hoga.

Day 4: AWS Networking + IAM

Topics + Deep Concept Notes

• VPC Architecture

VPC = isolated cloud network. Default /16 (65k IPs). Subnets /20-/28. **Public subnet:** route table mein IGW (Internet Gateway) entry. **Private subnet:** internet access NAT Gateway ke through. **Production pattern:** 2 public + 2 private subnets across 2 AZs. ALB public mein, EKS nodes private mein.

• Security Groups vs NACLs

SG (Stateful): Instance level, return traffic auto-allowed, only allow rules. **NACL (Stateless):** Subnet level, separate inbound/outbound rules, allow + deny. **Interview gotcha:** SG mein outbound 443 allow → response automatically aata hai. NACL mein outbound + inbound dono allow karne padte hain. NACL mostly default chhodte hain, SG actively use karte hain.

• VPC Endpoints

Gateway Endpoint: S3 + DynamoDB ke liye, free, route table entry. **Interface Endpoint (PrivateLink):** Other AWS services, paisa lagta hai, ENI banti hai. **Use case:** Private subnet se S3 access bina internet ke.

• IAM Deep

User vs Role: User = long-term credentials (humans), Role = temporary credentials (services, cross-account). **Policy types:** Identity-based (attach to user/role), Resource-based (S3 bucket policy), SCP (org-level guardrails). **Trust policy:** Kaun assume kar sakta hai role ko. **Permission policy:** Role kya kar sakta hai.

• IRSA (IAM Roles for Service Accounts)

Critical for EKS — tu use karta hai. Pod ko AWS API access dene ka secure way. **Flow:** EKS OIDC provider → ServiceAccount annotated with role ARN → pod mein env vars + token mount → AWS SDK auto-uses → STS AssumeRoleWithWebIdentity. **Why better than node IAM role:** Pod-level granularity, no shared permissions across pods, audit-friendly.

Resources (in order)

- **YouTube:** 'AWS VPC Core Concepts' by Stephane Maarek (free) — <https://www.youtube.com/c/StephaneMaarek>
- **YouTube:** 'AWS IAM Tutorial' by Be A Better Dev — <https://www.youtube.com/c/BeABetterDev>
- **AWS Docs:** <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>
- **IRSA Guide:** <https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html>
- **IRSA Blog:** <https://aws.amazon.com/blogs/opensource/introducing-fine-grained-iam-roles-service-accounts/>
- **Free practice Qs:** <https://tutorialsdojo.com> (SAA level)

Hands-on Practice

- AWS console mein VPC banao: 2 public + 2 private subnets, IGW, NAT GW. Diagram draw kar.
- EC2 launch private subnet mein, bastion host se SSH (public subnet through).
- IAM role banao with S3 read-only, CLI se aws sts assume-role test kar.

- EKS cluster pe IRSA setup kar (Minikube nahi, real EKS), pod se S3 list karo.
- **Day end:** NAT Gateway DESTROY kar — bill aata hai \$0.045/hour (\$32/month idle).

End-of-day Deliverable

Hand-drawn VPC architecture diagram + 'IRSA setup ke 5 steps' likha hua note.

Day 5: AWS Compute + Storage + EKS Deep

Topics + Deep Concept Notes

• EC2 + Storage

Instance types: t (burstable), m (general), c (compute), r (memory), p/g (GPU). Naam yaad rakh, deep nahi. **EBS vs EFS vs S3:** EBS = block storage (single AZ, EC2 attached), EFS = NFS (multi-AZ shared), S3 = object (URL-accessed). **EBS volume types:** gp3 (default, sasta), io2 (high IOPS), st1 (throughput-optimized HDD).

• S3 Deep

Storage classes: Standard → Standard-IA → One Zone-IA → Glacier Instant → Glacier Flexible → Deep Archive. Cost decreases, retrieval time increases. **Lifecycle policy:** 30 din baad IA, 90 din baad Glacier, 365 din baad delete. **Versioning:** on karo, accidental delete protection. **Encryption:** SSE-S3 (AWS managed key), SSE-KMS (customer managed), SSE-C (customer provided).

• S3 Access Control

3 mechanisms: Bucket policy (resource-based), IAM policy (identity-based), ACL (legacy, avoid). **Public bucket:** Block Public Access settings 4 hain — ye OFF na karein production mein. **Pre-signed URL:** Temporary public access without making bucket public.

• EKS Architecture Deep

Control plane: AWS managed (API server, etcd, scheduler) — \$73/month per cluster. **Data plane:** Tu manage karta hai — Managed Node Groups, Fargate, Self-managed EC2. **Managed Node Groups:** EC2 instances, AWS handles AMI updates, autoscaling. **Fargate:** Serverless pods, no node management, slightly costly, daemon sets nahi chalti.

• EKS Networking

VPC CNI (default): Pod gets VPC IP directly — koi NAT nahi, fast, but ENI limit per instance type. **Example:** m5.large = 3 ENIs × 10 IPs = 29 pods max. **Alternative:** Cilium CNI (eBPF-based, better perf + network policy).

• Cluster Autoscaler vs Karpenter

CA: Node groups expand/shrink, slow (2-10 min), needs ASG, single instance type per group. **Karpenter:** Node provisioning direct (no ASG), fast (30 sec), multi-instance types in single config, spot + on-demand mix. **Trending in 2025-26 — interview mein zaroor aayega.**

Resources (in order)

- **YouTube:** 'Amazon EKS Workshop' by AWS — search YouTube
- **YouTube:** 'Karpenter vs Cluster Autoscaler' by AWS Containers — <https://www.youtube.com/c/awscontainers>
- **EKS Best Practices:** <https://aws.github.io/aws-eks-best-practices/> (gold standard)
- **S3 Mastery:** 'Mastering S3' by Be A Better Dev (YouTube)
- **Karpenter docs:** <https://karpenter.sh/docs/>
- **EKS docs:** <https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html>

Hands-on Practice

- KillerCoda pe 'Amazon EKS' scenario kholo — 1 hour cluster free.
- Real EKS bana eksctl create cluster --name test --nodes 2, \$1 ka kharch, 1 ghanta use, destroy.
- Karpenter install kar, ek deployment scale karke node provisioning dekho.
- S3 bucket banao with versioning + lifecycle (30 din baad IA), pre-signed URL generate kar.

End-of-day Deliverable

3 questions ka jawab: (1) Pod ko external traffic kaise milta hai (LB → SVC → Pod full flow), (2) Node group upgrade strategy 5 steps, (3) Karpenter vs CA — 3 differences.

Day 6: AWS Specialized + Azure Refresh

Topics + Deep Concept Notes

• Parameter Store vs Secrets Manager

Parameter Store (free tier): Standard params free, advanced params \$0.05/10k. SecureString uses KMS. **Secrets Manager:** \$0.40/secret/month + \$0.05/10k API calls. Auto-rotation built-in. **Decision:** Static config + small secrets → Parameter Store. RDS/Aurora passwords with auto-rotation → Secrets Manager. **Tu use karta hai SSM Parameter Store** (Compliance Innovation mein) — articulate kar interview mein.

• External Secrets Operator (ESO) — Tera Resume Item

What: Kubernetes secrets sync from external sources (AWS SSM, Vault, GCP Secret Manager). **Components:** SecretStore (provider config), ExternalSecret (defines what to sync), ServiceAccount (with IAM role). **Flow:** ExternalSecret CRD → ESO controller → fetches from SSM → creates K8s Secret → pod mounts. **Refresh:** refreshInterval (default 1 hour) — secrets auto-update. **Why useful:** No secret in Git, no manual sync, audit trail.

• CloudWatch Basics

Logs: Log groups → log streams → log events. **CW Logs Insights query:** fields @timestamp, @message | filter @message like /ERROR/ | sort @timestamp desc. **Metrics:** namespaced (AWS/EC2, AWS/EKS), dimensions (per-instance), 1-min granularity (paid for sub-min). **Alarms:** Threshold-based, can trigger SNS, Auto Scaling, Lambda.

• RDS Multi-AZ vs Read Replica

Multi-AZ: Synchronous replica in different AZ for HA, automatic failover (60-120 sec), no read scaling. **Read Replica:** Asynchronous (lag possible), used for read scaling, can be different region. **Combine:** Production = Multi-AZ primary + 2-3 read replicas.

• Azure Refresh — AZ-204 Quick Hits

AKS: EKS equivalent, control plane free (unlike EKS), Azure CNI vs Kubenet. **Azure AD integration:** RBAC tied to AAD users — cleaner than EKS+IAM. **ACR:** Azure Container Registry, integrated with AKS via managed identity. **Key Vault:** Azure's secrets manager, integrates via CSI driver. **Azure DevOps:** tu strong hai isme — pipelines, service connections, agent pools.

Resources (in order)

→ **SSM Parameter Store:**

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>

→ **ESO:** <https://external-secrets.io> (architecture page must read)

→ **YouTube:** 'AZ-204 Crash Course' by John Savill (6 hr, sirf relevant 2 hr) — <https://www.youtube.com/c/NTFAQGuy>

→ **AKS docs:** <https://learn.microsoft.com/azure/aks/>

→ **CloudWatch Logs Insights:**

https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/CWL_QuerySyntax.html

Hands-on Practice

- Parameter Store mein SecureString banao, EC2 se IAM role use karke fetch karo.
- Minikube pe ESO install kar, fake provider se ek secret sync karke dekho.
- Azure free trial active hai? AKS cluster banao 1 ghanta, destroy.
- Azure DevOps pipeline YAML likho: Docker build → ACR push → AKS deploy.

End-of-day Deliverable

Notes: (1) Parameter Store vs Secrets Manager — 4 diff, (2) ESO 4-step flow, (3) Azure DevOps vs GitHub Actions — kab kya.

Day 7: AWS Practical + GCP Touch

Topics + Deep Concept Notes

• Common Architecture Scenarios

HA Web App: Route53 → CloudFront → ALB → ASG of EC2/EKS pods → RDS Multi-AZ + ElastiCache. **Cost optimization:** Right-sizing (CloudWatch metrics analyze), Spot for stateless, RIs/Savings Plans for steady, S3 lifecycle. **Multi-region DR:** Active-active (most expensive, no downtime), Active-passive (warm standby), Pilot light (minimal infra in DR), Backup-restore (cheapest, slowest).

• AWS Well-Architected Framework

6 pillars (yaad rakh): (1) Operational Excellence, (2) Security, (3) Reliability, (4) Performance Efficiency, (5) Cost Optimization, (6) Sustainability. Interview mein 'kaise design karoge' question pe pillar mention kar — points milte hain.

• Cost Optimization Levers

Compute: Spot (90% off, but interruption), Savings Plans (1/3 yr commit, 30-50% off), RIs (legacy SP). **Storage:** S3 lifecycle, EBS gp3 over gp2 (cheaper + better), delete unattached EBS. **Network:** NAT Gateway expensive (\$0.045/hr + data) — VPC endpoints for S3/DDB. **Tools:** Cost Explorer, Trusted Advisor, AWS Compute Optimizer.

• GCP Touch — GKE Basics

GKE Standard vs Autopilot: Autopilot = serverless K8s, pay per pod (sasta for small workloads). **Workload Identity:** IRSA equivalent — KSA → GSA mapping. **Cloud SQL:** Managed Postgres/MySQL, similar to RDS.

• Cross-Cloud Service Mapping

Compute: EC2 = Azure VM = Compute Engine. **K8s:** EKS = AKS = GKE. **Storage:** S3 = Blob = Cloud Storage. **Identity:** IAM = AAD = IAM. **Secrets:** SSM/SM = Key Vault = Secret Manager. **Functions:** Lambda = Azure Functions = Cloud Functions.

Resources (in order)

→ **YouTube:** 'AWS Solutions Architect Scenarios' by Be A Better Dev

→ **Well-Architected Whitepaper:** <https://aws.amazon.com/architecture/well-architected/>

→ **GKE in 100s:** <https://www.youtube.com/watch?v=jokQTW1JJD4> (Fireship)

→ **GKE docs:** <https://cloud.google.com/kubernetes-engine/docs>

→ **Cross-cloud cheatsheet:** Search 'AWS Azure GCP cheatsheet' on Google

Hands-on Practice

- Paper pe ek scenario design kar: 'E-commerce, 100K users daily, IN + US'. Full architecture diagram banao.
- GCP free tier active kar, ek GKE Autopilot cluster banao 1 ghanta, destroy.
- Ek Terraform script likho jo same VPC AWS + GCP banaye (multi-cloud comfort).

End-of-day Deliverable

1-page architecture diagram + cross-cloud comparison (EKS vs AKS vs GKE — 3 diff each).

PHASE 3: Kubernetes Deep (Day 8-10)

Goal: CKA hai, foundation strong hai. Ab production scenarios + troubleshooting pe focus.

Day 8: K8s Core + Pod Lifecycle

Topics + Deep Concept Notes

• Pod Lifecycle Phases

Pending: scheduler ne abhi place nahi kiya, ya image pull ho raha hai. **Running:** at least 1 container running. **Succeeded:** all containers exited 0 (Jobs ke liye). **Failed:** at least 1 container exited non-zero. **Unknown:** kubelet unreachable. **Common errors:** ImagePullBackOff, CrashLoopBackOff, OOMKilled, Evicted, Pending (no resources).

• Init Containers vs Sidecars

Init container: Pod start hone se PEHLE chalta hai, sequential, must complete. Use case: DB migration, wait-for service, config download. **Sidecar:** Main container ke saath chalta hai. Use case: Logging agent (Fluentd), service mesh proxy (Envoy/Istio), metric exporter.

• Probes — Critical Topic

Liveness: 'app alive hai?' Fail → kubelet RESTARTS container. **Readiness:** 'traffic accept kar sakta hai?' Fail → service endpoint se REMOVE (no restart). **Startup:** Slow-starting apps ke liye. Pass hone tak liveness/readiness disabled. **Interview gotcha:** Liveness aggressive mat banao — endless restart loop. Readiness pe aggressive ok.

• Resource Requests vs Limits

Request: Scheduler dekhta hai, guaranteed allocation. **Limit:** Hard ceiling, exceed pe CPU throttle, memory pe OOMKill. **QoS classes:** **Guaranteed** (req=limit for both CPU+mem) — last to be evicted. **Burstable** (req<limit) — middle priority. **BestEffort** (no req/limit) — first to evict.

• Scheduling Controls

nodeSelector: Simple key-value match. **Affinity (preferred over nodeSelector):** nodeAffinity (which nodes), podAffinity (co-locate with pods), podAntiAffinity (spread across nodes). **Taints + Tolerations:** Node has taint, only tolerating pods land there (e.g., GPU nodes). **Topology Spread:** Spread pods across AZs/nodes for HA.

• Workload Types Decision Tree

Stateless app: Deployment. **Stateful (DB, queue):** StatefulSet (stable network ID, ordered). **Per-node agent (logging, monitoring):** DaemonSet. **One-time task:** Job. **Scheduled task:** CronJob.

Resources (in order)

- **YouTube:** 'K8s Tutorial for Beginners' by TechWorld with Nana — <https://www.youtube.com/watch?v=X48VuDVv0do> (4hr, 2x speed = 2hr)
- **K8s docs (must):** <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>
- **Probes guide:** <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>
- **QoS docs:** <https://kubernetes.io/docs/tasks/configure-pod-container/quality-service-pod/>
- **KodeKloud CKA:** <https://kodekloud.com/courses/certified-kubernetes-administrator-cka/> (paid, but YouTube playlist free)

Hands-on Practice

- Minikube pe pod with 3 probes — intentionally fail kar, behavior observe kar.

- Deployment 5 replicas + PDB minAvailable=3, kubectl drain test.
- StatefulSet with PVC — delete + recreate, data persist hai check.
- kubectl explain pod.spec.affinity — har field samajh.

End-of-day Deliverable

Notes: (1) Liveness vs Readiness swap effect, (2) StatefulSet vs Deployment 4 diff, (3) Pod Pending — 5 debug steps.

Day 9: K8s Networking + Storage + Security

Topics + Deep Concept Notes

• Service Types

ClusterIP (default): Cluster-internal IP, only accessible within cluster. **NodePort:** Exposes on every node's IP at static port (30000-32767). For dev/testing. **LoadBalancer:** Cloud provider creates external LB (AWS ALB/NLB, Azure LB). **Headless (clusterIP: None):** No load balancing, returns pod IPs directly via DNS. Used by StatefulSets. **ExternalName:** CNAME to external service (database.example.com).

• Ingress vs Gateway API

Ingress: Old way, vendor-specific annotations (NGINX, ALB, Traefik), L7 routing. **Gateway API:** New standard (2023+), role-based (GatewayClass for cluster admin, Gateway for infra team, HTTPRoute for app team), better multi-tenancy, vendor-neutral. **2025-26 trend — interview mein zaroor aata hai.**

• NGINX Ingress (Tu Use Karta Hai)

Annotations gold: nginx.ingress.kubernetes.io/rewrite-target, /backend-protocol, /rate-limit, /enable-cors. **Path types:** Exact, Prefix, ImplementationSpecific. **TLS:** tls section in spec, secretName references K8s secret with cert+key.

• Network Policies

Default: All pods can talk to all pods (open). **NetworkPolicy:** Whitelist-based — once applied to pod, only allowed traffic flows. **Production pattern:** Default-deny + explicit allow rules. **Requires CNI support:** Calico, Cilium support. AWS VPC CNI by default DOESN'T (need to enable).

• Service Mesh Concept

What: Sidecar proxy (Envoy) per pod handles all network. **Features:** mTLS automatic, traffic shaping (canary), circuit breaking, observability (traces). **Istio vs Linkerd:** Istio = feature-rich, complex. Linkerd = simpler, faster. **Trade-off:** Resource overhead (CPU+memory per pod) vs features.

• Storage: PV, PVC, StorageClass

PV: Cluster-level storage resource (admin creates). **PVC:** Pod-level claim (user creates). **StorageClass:** Dynamic provisioning template — PVC creates PV automatically. **Reclaim policy:** Retain (keep data), Delete (remove with PVC), Recycle (deprecated). **AWS:** EBS CSI for block, EFS CSI for shared.

• RBAC Deep

Subjects: User, Group, ServiceAccount. **Resources:** pods, services, deployments, etc. + verbs (get, list, create, delete). **Role:** namespace-scoped permissions. **ClusterRole:** cluster-wide. **RoleBinding:** Role → subject in namespace. **ClusterRoleBinding:** ClusterRole → subject cluster-wide. **Best practice:** Least privilege, ServiceAccount per app.

Resources (in order)

→ **YouTube:** 'K8s Networking Explained' by IBM Cloud — <https://www.youtube.com/watch?v=5cNrTU6o3Fw>

- **Ingress docs:** <https://kubernetes.io/docs/concepts/services-networking/ingress/>
- **Gateway API:** <https://gateway-api.sigs.k8s.io>
- **Network Policy editor:** <https://editor.networkpolicy.io> (game changer visual tool)
- **RBAC YouTube:** 'K8s RBAC' by That DevOps Guy — <https://www.youtube.com/c/MarcelDempers>
- **NGINX Ingress annotations:**
<https://kubernetes.github.io/ingress-nginx/user-guide/nginx-configuration/annotations/>

Hands-on Practice

- Minikube pe NGINX Ingress install, 2 services route (path-based + host-based).
- Network policy banao — namespace isolation. curl pods se test kar.
- ServiceAccount + Role with limited perms, kubectl --as se test.
- ESO with fake provider, secret sync hote dekho.

End-of-day Deliverable

Architecture diagram: 'Internet request → Pod tak full flow' (DNS → LB → Ingress → SVC → Pod).

Day 10: Helm + Troubleshooting + Production Patterns

Topics + Deep Concept Notes

• Helm Chart Anatomy

Chart.yaml: metadata (name, version, appVersion, dependencies). **values.yaml:** default config — user overrides karta hai. **templates/:** Go templates jo K8s YAML banate hain. **_helpers.tpl:** reusable template functions ({{ include "app.fullname" . }}). **charts/:** sub-charts (dependencies). **NOTES.txt:** install ke baad user ko message.

• Helm Templating Tricks

Pipelines: {{ .Values.image.tag | default "latest" | quote }} **Conditionals:** {{- if .Values.ingress.enabled }} ... {{- end }} **Loops:** {{- range .Values.envVars }} **Named templates:** {{ define "app.labels" }} ... {{ end }} **{{- vs {{:** Dash trims whitespace.

• Helm Lifecycle + Hooks

install → **upgrade** → **rollback** → **uninstall**. **Hooks:** pre-install, post-install, pre-upgrade, post-upgrade, pre-delete (annotations on resources). **Use case:** DB migration as pre-upgrade Job.

• Pulumi vs Helm — Tu Dono Use Karta Hai

Helm: Templated YAML, Go templates, packaged charts. Best for: distributing apps, simple deployments. **Pulumi:** Real programming languages (TS/Python/Go), full logic, type safety. Best for: complex infra, multi-cloud, conditional logic. **Pulumi Operator:** Run Pulumi inside K8s — GitOps style. **Articulation:** 'Helm for app deploys, Pulumi for infra + cross-cutting concerns.'

• K8s Troubleshooting Framework

Step 1: kubectl get pods -A — overview. **Step 2:** kubectl describe pod — events section gold hai. **Step 3:** kubectl logs [--previous] — app errors. **Step 4:** kubectl exec -it -- /bin/sh — inside investigate. **Step 5:** kubectl get events --sort-by='.lastTimestamp' — cluster events.

• Common Errors + Diagnosis

ImagePullBackOff: Image registry auth, image name typo, network. **CrashLoopBackOff:** App crashes immediately — check logs (--previous). **Pending:** No node has resources, PVC not bound, taint without toleration. **OOMKilled:** Memory limit too low. **Evicted:** Node pressure (disk/memory) — pod kicked out.

- **Deployment Strategies**

Rolling Update (default): maxSurge + maxUnavailable controls. Slow, safe. **Blue-Green:** 2 envs, switch traffic at LB level. Fast rollback. **Canary:** 5% traffic to new, monitor, gradually 100%. Argo Rollouts/Flagger automate. **A/B testing:** Header-based routing, separate from canary.

Resources (in order)

- **YouTube:** 'Helm Tutorial' by TechWorld with Nana — <https://www.youtube.com/watch?v=-ykwb1d0DXU>
- **Helm docs (must):** https://helm.sh/docs/chart_template_guide/
- **YouTube:** 'K8s Troubleshooting' by Sysdig
- **kubectl debug cheatsheet:** <https://learnk8s.io/troubleshooting-deployments> (poster gold)
- **KillerCoda:** 'K8s Troubleshooting' scenarios (free)
- **Argo Rollouts:** <https://argoproj.github.io/argo-rollouts/>

Hands-on Practice

- Custom Helm chart scratch se — webapp + service + ingress.
- Chart break karo intentionally: galat image, OOM trigger. kubectl describe se debug.
- Conditional logic in Helm: env-specific config (if dev/prod).
- helm rollback 3 versions ke beech mein.

End-of-day Deliverable

Personal K8s Troubleshooting Cheatsheet — top 10 errors + 3-step diagnosis. Interview ke 1 ghanta pehle revise karega.

PHASE 4: IaC + CI/CD (Day 11-12)

Day 11: Terraform + Pulumi Deep

Topics + Deep Concept Notes

• Terraform State Management

Local state (dev only): terraform.tfstate file. NEVER commit to Git (has secrets). **Remote backend:** S3 (storage) + DynamoDB (locking) — production standard. **Why locking:** Concurrent terraform apply ko prevent karta hai (state corruption avoid). **Backend config:** backend "s3" block in terraform{} — bucket, key, region, dynamodb_table.

• Workspaces vs Separate State

Workspaces: Same code, different state per workspace (dev/stage/prod). Cheap, but risk of mistakes (terraform.workspace var). **Separate state files:** Different directories per env, more isolation, more code duplication. **Recommended:** Separate state files for prod isolation, modules for code reuse.

• Modules — Reusability

Structure: main.tf, variables.tf, outputs.tf, README. **Versioning:** Git tags (v1.2.0), Terraform Registry, or internal module registry. **Source:** source = "git::https://github.com/org/modules.git/vpc?ref=v1.2.0". **Best practice:** Pin module versions, never use main branch in prod.

• Terraform Commands Power Users Know

terraform import aws_instance.web i-1234abcd — manually-created resource ko state mein lao. **terraform state mv aws_instance.old aws_instance.new** — refactor without recreate. **terraform state rm aws_instance.web** — state se hata, but resource live rahe. **terraform refresh** — state ko real infra se sync. **terraform taint / -replace** — force recreation next apply.

• Drift Detection

What: Terraform-managed infra ko manually console mein modify kar diya — state ↔ reality mismatch. **Detection:** terraform plan in CI/CD daily — non-zero changes alert. **Tools:** driftctl (free), Atlantis (PR-based), Spacelift/env0 (paid). **Prevention:** SCP/IAM policies block manual console changes for terraform-managed resources.

• Pulumi Deep

Why Pulumi over Terraform: Real programming = loops, conditionals, abstraction (classes), tests, IDE intellisense. **State backend options:** Pulumi Cloud (free for individuals), AWS S3, Azure Blob, self-hosted. **Stacks:** dev/stage/prod isolation, pulumi config set per stack. **Pulumi Operator (tu use karta hai):** Pulumi runs inside K8s as CRD-driven — GitOps for infra.

• When to Use Each

Terraform: Stable team, declarative preference, large community, simple infra. **Pulumi:** Programming preference, complex logic, multi-cloud abstractions, test infra code. **Tu jo bolega interview mein:** 'TokenTide pe Pulumi use kar raha hu kyunki Dgraph deployment mein conditional logic + custom resources thi. Cybage pe Terraform tha kyunki team familiar.'

Resources (in order)

→ **YouTube:** 'Terraform Tutorial' by TechWorld with Nana — https://www.youtube.com/watch?v=l5k1ai_GBDE

→ **Best Practices:** <https://www.terraform-best-practices.com>

→ **Pulumi Concepts:** <https://www.pulumi.com/docs/concepts/>

→ **Pulumi vs TF:** <https://www.pulumi.com/docs/intro/vs/terraform/>

→ **Book (free Ch):** 'Terraform Up & Running' — <https://www.terraformupandrunning.com>

→ **driftctl:** <https://driftctl.com>

Hands-on Practice

- Terraform module: VPC + EKS + node group. Reusable banao.
- Remote backend setup: S3 + DynamoDB. Apply karo, state remote pe verify.
- terraform import manually-created resource ko.
- Pulumi stack TypeScript/Python mein same VPC.
- Drift simulate: AWS console se resource modify, terraform plan dekho.

End-of-day Deliverable

5 Q&A:: state corrupt fix, Pulumi vs TF when, drift detection, module versioning, multi-env structure.

Day 12: GitHub Actions + Azure DevOps

Topics + Deep Concept Notes

• GitHub Actions Anatomy

Workflow: .github/workflows/*.yaml. **Triggers:** on: push, pull_request, schedule, workflow_dispatch (manual), workflow_call (reusable). **Jobs:** parallel by default, dependencies via 'needs:'. **Steps:** sequential within job. **Runners:** ubuntu-latest (free, GitHub-hosted), self-hosted (security, custom hardware).

• Reusable Workflows + Composite Actions

Reusable workflow: on: workflow_call, called from another workflow. Whole workflow reuse. **Composite action:** action.yaml with multiple steps, called as single step. Atomic reuse. **Use case:** Build-and-push Docker workflow used by 20 repos.

• Matrix Builds + Conditional Execution

Matrix: Run job multiple times with different inputs (Node 18/20/22, OS, etc.). **If conditions:** if: github.ref == 'refs/heads/main'. **Strategy:** fail-fast (default true) — one fails, all cancel.

• Secrets Management — OIDC (Modern, 2025-26)

Old way: AWS_ACCESS_KEY_ID + SECRET in GitHub secrets. Long-lived, leak risk. **OIDC way:** GitHub issues short-lived JWT, AWS verifies, gives temporary STS credentials. NO long-lived keys. **Setup:** AWS IAM OIDC provider for token.actions.githubusercontent.com → IAM role with trust policy → workflow uses configure-aws-credentials@v4 with role-to-assume. **Interview gold:** Articulate this — top product companies expect OIDC.

• Caching + Artifacts

actions/cache: Speed up builds (npm, pip, Docker layers). Key based on lock file hash. **actions/upload-artifact + download-artifact:** Pass files between jobs. **Docker layer caching:** docker/build-push-action with cache-from + cache-to.

• Azure DevOps YAML Pipelines

Structure: stages → jobs → steps. trigger:, pool:, variables:. **Service connections:** Cloud auth (AWS/Azure/GCP), Docker registry, GitHub. **Variable groups:** Shared vars across pipelines. Linked to Key Vault for secrets. **Environments + approvals:** Manual approval gates between stages. **Agent pools:** Microsoft-hosted, self-hosted, scale-set.

• Deployment Strategies in Pipelines

Blue-green: Deploy to inactive env, switch LB. Quick rollback. **Canary:** Argo Rollouts in K8s, gradual traffic shift. **Rolling:** K8s default, maxSurge/maxUnavailable. **Feature flags:** Code deployed but feature toggled (LaunchDarkly, Unleash) — separate deploy from release.

- **Branching Strategy**

GitFlow (legacy): main, develop, feature/*, release/*, hotfix/*. Complex, slow. **Trunk-based (modern):** main + short-lived feature branches (1-2 days). Continuous integration. **2025-26 standard:** Trunk-based for most teams.

Resources (in order)

- **YouTube:** 'GitHub Actions Tutorial' by TechWorld with Nana — https://www.youtube.com/watch?v=R8_veQiYBjI
- **OIDC Guide (must):** <https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-amazon-web-services>
- **Azure DevOps:** 'YAML Pipelines Crash Course' by Adam Marczak — <https://www.youtube.com/c/MarczakIO>
- **Reusable Workflows:** <https://docs.github.com/actions/using-workflows/reusing-workflows>
- **Awesome Actions:** <https://github.com/sdras/awesome-actions>
- **Trunk-based:** <https://trunkbaseddevelopment.com>

Hands-on Practice

- Workflow likho: code → test → Docker build → ECR push → EKS deploy. OIDC use kar.
- Reusable workflow banao, second repo se call kar.
- Matrix build: 3 Node versions parallel test.
- Azure DevOps free tier mein YAML pipeline with approval gate.

End-of-day Deliverable

Production-grade GitHub Actions workflow YAML (apna) — interview mein dikhane ke kaam aayega. Plus OIDC 5-step note.

PHASE 5: Observability + Debugging (Day 13)

Day 13: Prometheus, Grafana, OTel, Logging

Topics + Deep Concept Notes

• Three Pillars

Metrics: Numbers over time (CPU%, request rate, latency p99). Cheap to store, aggregated. **Logs:** Discrete events with context (error stack traces). Expensive at scale. **Traces:** Request journey across services. Critical for microservices.

• Prometheus Architecture

Pull model: Prometheus server scrapes /metrics endpoint of targets. **Service discovery:** Static config, file_sd, DNS, Kubernetes API (auto-discover pods). **TSDB:** Time-series database, on-disk, configurable retention (default 15 days). **Push gateway:** For short-lived jobs (Cron) jo pull se nahi reach hote.

• PromQL Basics

Counter: Always increases (request count). Use rate()/increase(). **Gauge:** Goes up-down (memory usage, queue size). Use directly. **Histogram:** Buckets (latency distribution). Use histogram_quantile(0.99, rate(http_duration_bucket[5m])). **Common queries:** rate(http_requests_total[5m]) — req/sec. sum by (service) (rate(errors[5m])) — error rate per service. histogram_quantile(0.99, rate(latency_bucket[5m])) — p99 latency.

• Cardinality Explosion — Critical

What: Each unique label combination = new time series. Example: HTTP method (5) × status (10) × endpoint (100) = 5000 series. Add user_id (10K users) = 50M series → DB crash. **Avoid:** High-cardinality labels (user_id, session_id, request_id) in metrics. Put them in logs instead. **Detect:** Prometheus topk(10, count by (__name__){__name__=~".+"})).

• Grafana — Tu Use Karta Hai

Data sources: Prometheus, Loki, CloudWatch, Tempo (traces), MySQL. Single dashboard, multiple sources. **Variables:** Dropdown filters (env, service). \$variable in queries. **Alerting:** Unified alerting (since v8) — define alerts in Grafana, route to Slack/PagerDuty. **Best practice:** Dashboards as code — JSON in Git, version-controlled.

• RED Method vs USE Method

RED (request-based, services): Rate (req/sec), Errors (errors/sec), Duration (p50/p99 latency). **USE (resource-based, infra):** Utilization (% busy), Saturation (queue depth), Errors (counts). **When:** RED for app services, USE for hosts/databases/network.

• Logging — Loki vs ELK vs CloudWatch

Loki: Indexes only labels (not log content) — cheap. Best with Grafana. PromQL-like syntax (LogQL). **ELK (Elasticsearch + Logstash + Kibana):** Full-text indexing, powerful search, expensive at scale. **CloudWatch Logs:** Easy on AWS, expensive (\$0.50/GB ingestion), slow queries on large data. **Decision:** K8s + cost-conscious → Loki. Need full-text search + complex queries → ELK. AWS-native, low volume → CW.

• Structured Logging

Plain log: 'User 1234 failed login at 10:30 from 1.2.3.4' — hard to query. **JSON log:** {"user_id":1234,"event":"login_failed","ip":"1.2.3.4","ts":...} — queryable. **Levels:** DEBUG (dev), INFO (default), WARN, ERROR, FATAL. Production = INFO+. **Sampling:** High-volume logs sample (1 in 100) to reduce cost.

• OpenTelemetry — Modern Standard

Why: Vendor-neutral instrumentation. Same code → ship to Datadog, Jaeger, NewRelic, anywhere.

Components: SDK (in app code), Collector (separate service), Exporters (to backend). **Collector**

pipeline: Receivers (OTLP, Jaeger, Prometheus) → Processors (batch, sampling, filtering) → Exporters (Tempo, Datadog, etc.). **Auto-instrumentation:** Java/Python/Node agents — no code change.

• Distributed Tracing

Trace: End-to-end journey of a request (e.g., API gateway → service A → service B → DB). **Span:** Single operation in trace (one HTTP call, one DB query). **TraceID propagation:** W3C TraceContext header (traceparent) through all services. **Tools:** Jaeger (open-source), Tempo (Grafana stack), Datadog APM, New Relic.

• Debugging Methodology — Production Slow API

Step 1: Confirm scope (one endpoint? all? specific user?). **Step 2:** Check dashboards — RED metrics. Latency p99 spike kahan? **Step 3:** Trace a slow request — which span is slow? **Step 4:** If DB span — check slow query logs. If external API — check upstream status. **Step 5:** If app span — check pod resources (CPU throttling? GC pauses?). **Step 6:** Correlate with deployments — was anything pushed in last hour? **Step 7:** Mitigate (rollback / scale up) → then investigate root cause.

Resources (in order)

- **YouTube:** 'Prometheus Monitoring' by TechWorld with Nana — <https://www.youtube.com/watch?v=h4Sl21AKiDg>
- **YouTube:** 'Grafana Tutorial' by Anton Putra
- **SRE Book Ch 6 (FREE):** <https://sre.google/sre-book/monitoring-distributed-systems/>
- **OpenTelemetry:** <https://opentelemetry.io/docs/concepts/>
- **PromQL Cheatsheet:** <https://promlabs.com/promql-cheat-sheet/>
- **RED Method:** <https://grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-services/>
- **USE Method:** <https://www.brendangregg.com/usemethod.html>
- **Loki:** <https://grafana.com/docs/loki/latest/>

Hands-on Practice

- Minikube pe kube-prometheus-stack Helm install (Prometheus + Grafana + Alertmanager).
- 5 PromQL: pod CPU, error rate, p99 latency, req rate per service, memory trend.
- Grafana dashboard with 4 panels — RED method.
- OTel collector deploy, sample app traces bhejo, Jaeger pe dekho.
- Apne TokenTide Grafana setup ki story refresh kar.

End-of-day Deliverable

Debugging Playbook — 'API latency spike — 7-step diagnosis'.

PHASE 6: Mock + Polish (Day 14-15)

Day 14: System Design + Mock Round 1

Topics + Deep Concept Notes

- DevOps System Design Framework

Step 1: Clarify Requirements — Functional (kya karna hai), Non-functional (scale, SLA, cost). **Step 2: Constraints** — Budget, team size, existing tech, compliance. **Step 3: High-level Architecture** — Boxes and arrows. Components naam kar. **Step 4: Deep Dive 1-2 components** — Trade-offs articulate kar. **Step 5: Scale + Failure** — 10x grow toh kya badlega? Single point of failure kahan? **Step 6: Monitoring + Cost** — How to operate, kitna paisa.

- Common Scenarios — Practice These

1. CI/CD for 50 microservices: GitHub Actions/Jenkins, mono-repo vs poly-repo, build matrix, deployment strategy, secrets, rollback. **2. Multi-region DR:** Active-active vs passive, data replication (sync vs async), DNS failover, RTO/RPO targets. **3. Observability for 1000 services:** Metric cardinality, log volume, sampling, cost, on-call routing. **4. Secrets at scale:** 1000 services × 5 envs = 5000 secrets. Vault/SSM, rotation, audit, K8s integration. **5. Zero-downtime deploys:** Health checks, graceful shutdown, draining connections, blue-green vs canary.

- Trade-off Thinking

Cost vs Performance: Spot instances cheap but interrupt. Multi-AZ doubles cost but adds HA. **Consistency vs Availability:** Multi-region writes — global lock (consistent slow) vs eventual (fast inconsistent). **Simplicity vs Flexibility:** Helm template (simple) vs Pulumi code (flexible). **Build vs Buy:** In-house Vault vs AWS Secrets Manager. Maintenance vs control.

Resources (in order)

- **YouTube:** Search 'DevOps System Design Interview' — top 3 videos
- **Blog:** 'How Discord stores billions of messages' — <https://discord.com/blog/how-discord-stores-billions-of-messages>
- **Stripe Engineering:** <https://stripe.com/blog/engineering>
- **Mock platform 1:** Pramp — <https://www.pramp.com> (peer-to-peer, free)
- **Mock platform 2:** Interviewing.io — <https://interviewing.io> (anonymous, free)

Hands-on Practice

- **Self-mock 75 min:** Phone record. Scenario uthao, 45 min loud bolke design. 30 min review apni recording.
- **Pramp/peer mock:** Aaj ya kal schedule kar.
- Apna system design template banao — flow yaad rakh.

End-of-day Deliverable

3 system design problems ka 1-page sketch (paper). 1 actual recorded mock done.

Day 15: Final Mock + Story Polish + Confidence

Topics + Deep Concept Notes

- Resume Story Tightening

2-min rule: Har story 2 min mein khatam ho. **Hook** → **Action** → **Impact:** First 15 sec mein context, fir details, last 30 sec impact + numbers. **Numbers add karna mandatory:** '50 services migrated', '\$X saved/month', 'reduced deployment time from 2 hours to 15 min'.

- **Tricky Questions Prep**

'Biggest failure?' — Real one (small scale), what learned, what changed. NEVER 'I'm a perfectionist'. **'Why leaving?'** — Growth, learning, scale, NOT 'manager bad' or 'team sucks'. **'Why hire you?'** — 3 specific things: (1) experience match, (2) hands-on production, (3) autonomy + can ship. **'5-year plan?'** — Tech depth (Staff/Principal SRE) ya breadth (Platform Engineering Lead) — koi bhi but specific.

- **Salary Negotiation Basics**

Don't share current salary first. 'Looking for market-aligned compensation given my experience'. **Ranges:** Senior DevOps in India 2025-26 = 18-45 LPA depending on company tier. **Components:** Base + Bonus + ESOPs/RSUs + Joining bonus. Negotiate ESOPs (often flex).

- **Questions to ASK the Interviewer**

Smart: 'Team ka biggest engineering challenge abhi kya hai?' **Strategic:** 'This role's success metrics 6 months mein kya honge?' **Casual:** 'Aapko kya pasand hai is team mein?' **Engineering culture:** 'On-call rotation kaise structured hai?' 'Tech debt vs new features ka split?' **NEVER ask:** Working hours, leave policy, salary in tech round (HR mein discuss).

- **Day-of-Interview Checklist**

Tech setup: Laptop charged, charger plugged, backup phone hotspot. **Audio/video:** Mic test, headphones ready, lighting front-facing. **Background:** Clean wall ya virtual background. **Notes:** Resume printed, water bottle, pen + paper. **Calm:** 30 min pehle deep breaths, light walk, no caffeine overdose.

Resources (in order)

- **The Pragmatic Engineer:** <https://blog.pragmaticengineer.com> (career advice gold)
- **Levels.fyi salary data:** <https://www.levels.fyi/?country=India>
- **Glassdoor company reviews:** <https://www.glassdoor.co.in>
- **YouTube:** 'Why are you leaving' answers — search Don Georgevich

Hands-on Practice

- **Final mock 90 min:** Pramp/peer — full round (behavioral + technical + design).
- 6 STAR stories revise, 2-min format bolke practice.
- 10 questions likh interviewer se poochne ke.
- **Day 16 ke liye 1-page final cheatsheet** — interview ke subah revise karega.

End-of-day Deliverable

Interview-ready confidence! + 1-page master cheatsheet for D-day.

10. Deep Concept Reference (Quick Lookup)

Ye section interview se 1 ghanta pehle bhi revise kar sakta hai. Topic-wise rapid-fire concepts.

Networking Quick Reference

- **OSI Layers:** 1 Physical, 2 Data Link, 3 Network (IP), 4 Transport (TCP/UDP), 5 Session, 6 Presentation, 7 Application (HTTP).
- **TCP 3-way handshake:** SYN → SYN-ACK → ACK. Connection established.
- **HTTP status:** 2xx success, 3xx redirect, 4xx client (401 auth, 403 forbidden, 404, 429 rate), 5xx server (500, 502 bad gw, 503 unavail, 504 gw timeout).
- **DNS records:** A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), TXT (verify/SPF), NS (nameserver), SOA (start of authority).
- **CIDR:** /24 = 256 IPs (254 usable), /16 = 65536, /20 = 4096, /28 = 16. Formula: $2^{(32-\text{prefix})}$.
- **L4 vs L7 LB:** L4 = TCP/UDP fast, L7 = HTTP-aware (path routing, WAF).

Kubernetes Quick Reference

- **Pod phases:** Pending → Running → Succeeded/Failed. Errors: ImagePullBackOff, CrashLoopBackOff, OOMKilled, Evicted.
- **Probes:** Liveness (restart on fail), Readiness (remove from service on fail), Startup (slow apps).
- **QoS:** Guaranteed (req=limit), Burstable (req<limit), BestEffort (no req/limit). Eviction order: BestEffort first.
- **Services:** ClusterIP (internal), NodePort (every node), LoadBalancer (external), Headless (returns pod IPs), ExternalName (CNAME).
- **Ingress vs Gateway API:** Ingress old, vendor-specific. Gateway API new (2023+), role-separated, vendor-neutral.
- **StatefulSet vs Deployment:** SS = stable network ID + persistent storage + ordered start/stop. D = stateless replicas.
- **RBAC:** Role (ns) + ClusterRole (cluster). Bound via RoleBinding + ClusterRoleBinding to User/Group/SA.
- **Network Policy:** Default = open. Apply policy = whitelist mode. Needs CNI support (Calico, Cilium).
- **HPA vs VPA vs CA vs Karpenter:** HPA = pod replicas, VPA = pod resources, CA = node group size, Karpenter = node provisioning direct.
- **Helm hooks:** pre-install, post-install, pre-upgrade, post-upgrade, pre-delete (annotation on K8s resource).

AWS Quick Reference

- **SG vs NACL:** SG stateful (instance), NACL stateless (subnet, allow+deny).
- **VPC Endpoints:** Gateway (S3, DDB, free), Interface (other svcs, paid).
- **IAM:** User (long-term), Role (temporary, for services). Trust policy = who can assume. Permission policy = what role can do.
- **IRSA:** EKS OIDC → SA annotated → pod gets STS token → AWS SDK auto-uses. Pod-level IAM.
- **S3 classes:** Standard → IA → One-Zone IA → Glacier Instant → Flexible → Deep Archive (cost ↓, retrieval time ↑).
- **Parameter Store vs Secrets Manager:** SSM = free standard, manual rotation. SM = \$0.40/secret + auto-rotation.

- **RDS Multi-AZ vs Read Replica:** Multi-AZ = sync HA (no read scale). RR = async, read scaling.
- **Karpenter vs CA:** Karpenter = direct node provision (fast, multi-instance), CA = ASG-based (slower, single type).
- **Well-Architected Pillars:** Operational Excellence, Security, Reliability, Performance, Cost, Sustainability.

Terraform / IaC Quick Reference

- **State backends:** S3 + DynamoDB locking (prod), local (dev only — never commit).
- **Workspaces:** Same code, different state per env. Cheap but risky for prod isolation.
- **terraform import:** Manually-created resource ko state mein lao without recreate.
- **terraform state mv/rm:** Refactor without recreating live resources.
- **Drift:** Manual changes outside TF. Detect via plan in CI, driftctl, Atlantis.
- **Pulumi vs TF:** Pulumi = real code (TS/Python/Go). TF = HCL declarative. Pulumi for complex logic, TF for simple infra + community modules.

CI/CD Quick Reference

- **OIDC (modern):** No long-lived AWS keys. GitHub issues JWT → AWS verifies → temp STS creds.
- **Reusable workflow:** on: workflow_call. **Composite action:** action.yml with steps.
- **Matrix build:** Multiple combinations parallel (Node 18/20/22 × OS).
- **Branching:** Trunk-based (modern, short-lived branches) vs GitFlow (legacy, complex).
- **Deployment strategies:** Rolling (default safe), Blue-green (fast switch), Canary (gradual + monitor), Feature flags (decouple deploy from release).

Observability Quick Reference

- **3 pillars:** Metrics (numbers, cheap), Logs (events, costly), Traces (request journey, microservices critical).
- **RED:** Rate, Errors, Duration (services). **USE:** Utilization, Saturation, Errors (resources).
- **Cardinality explosion:** High-card labels (user_id) in metrics = DB death. Put in logs instead.
- **Loki vs ELK vs CW:** Loki cheap (label-only index). ELK expensive but powerful. CW easy + AWS-native.
- **OpenTelemetry:** Vendor-neutral. SDK + Collector + Exporters. W3C TraceContext propagation.
- **SLI/SLO/SLA:** SLI = measurement, SLO = internal target, SLA = external contract. Error budget = 1 - SLO.

11. Master Resource List

Free YouTube Channels (Bookmark)

- **TechWorld with Nana** — DevOps fundamentals, K8s, TF — youtube.com/c/TechWorldwithNana
- **Anton Putra** — Modern DevOps + comparisons — youtube.com/@AntonPutra
- **Be A Better Dev** — AWS deep dives — youtube.com/c/BeABetterDev
- **John Savill's Tech Training** — Azure best — youtube.com/c/NTFAQGuy
- **Stephane Maarek** — AWS certs — youtube.com/c/StephaneMaarek
- **That DevOps Guy / Marcel Dempers** — Practical K8s — youtube.com/c/MarcelDempers
- **KodeKloud** — CKA/CKAD prep — youtube.com/@KodeKloud
- **Adam Marczak** — Azure DevOps — youtube.com/c/MarczakIO
- **Cloud With Raj** — Hindi-friendly Azure — youtube.com/@CloudWithRaj
- **Hussein Nasser** — Networking + Backend — youtube.com/@hnasr
- **Fireship** — 100s explainers — youtube.com/@Fireship

Documentation (Primary Sources)

- **Kubernetes:** kubernetes.io/docs/concepts/
- **EKS Best Practices:** aws.github.io/aws-eks-best-practices (gold)
- **Terraform:** developer.hashicorp.com/terraform/docs
- **Pulumi:** pulumi.com/docs
- **Helm:** helm.sh/docs/chart_template_guide/
- **Prometheus:** prometheus.io/docs/
- **OpenTelemetry:** opentelemetry.io/docs/concepts/
- **Grafana:** grafana.com/docs/grafana/latest/
- **SRE Book (FREE):** sre.google/books
- **AWS Architecture:** aws.amazon.com/architecture/
- **External Secrets Operator:** external-secrets.io
- **Karpenter:** karpenter.sh/docs/
- **Argo CD:** argo-cd.readthedocs.io
- **Cloudflare Learning:** cloudflare.com/learning/

Practice Platforms (Hands-on)

- **KillerCoda:** killercoda.com — Free K8s/Linux/Docker labs in browser
- **Play with K8s:** labs.play-with-k8s.com — 4-hour free sessions
- **Play with Docker:** labs.play-with-docker.com
- **LocalStack:** localstack.cloud — AWS locally simulate
- **KodeKloud Engineer:** kodekloud.com/engineer (real-world tasks, free tier)
- **Pramp:** pramp.com — Peer mock interviews, FREE
- **Interviewing.io:** interviewing.io — Anonymous mocks with engineers
- **tutorialsdojo:** tutorialsdojo.com — Free AWS practice questions
- **cidr.xyz:** Subnet calculator + practice

- **Network Policy editor:** editor.networkpolicy.io — Visual K8s NP

Books (Free chapters available)

- **The DevOps Handbook** by Gene Kim — culture + practices
- **Kubernetes Up & Running** — O'Reilly classic
- **Terraform Up & Running** by Yevgeniy Brikman — terraformupandrunning.com
- **Site Reliability Engineering** (Google, FREE) — sre.google/sre-book/
- **The SRE Workbook** (Google, FREE) — sre.google/workbook/
- **Designing Data-Intensive Applications** by Martin Kleppmann — distributed systems bible
- **System Design Interview** by Alex Xu — practical examples

Blogs to Follow

- **The Pragmatic Engineer:** blog.pragmaticengineer.com (career)
- **Stripe Engineering:** stripe.com/blog/engineering
- **Cloudflare Blog:** blog.cloudflare.com (network deep dives)
- **Discord Engineering:** discord.com/category/engineering
- **Netflix Tech Blog:** netflixtechblog.com
- **Uber Engineering:** uber.com/blog/engineering/
- **Meta Engineering:** engineering.fb.com
- **AWS Blog (Containers):** aws.amazon.com/blogs/containers/
- **Brendan Gregg:** brendangregg.com (perf debugging legend)

12. Top 50 Interview Questions

Day 14-15 mein in sabka short answer ready rakhna. 2-3 sentences ka jawab — interview mein time bachega.

Kubernetes (15 Qs)

1. Pod stuck in Pending — debug ke 5 steps?
2. Liveness vs Readiness probe — agar swap kar diye toh kya hoga?
3. StatefulSet vs Deployment — 4 key differences?
4. How does kube-scheduler decide pod placement?
5. Network policy default behavior — allow all ya deny all?
6. RBAC: Role vs ClusterRole — kab kya?
7. ConfigMap vs Secret — security difference?
8. Service ClusterIP vs NodePort vs LoadBalancer?
9. Ingress vs Gateway API — kyun Gateway API better?
10. PVC bound nahi ho raha — possible causes?
11. Pod OOMKilled — investigation steps?
12. Cluster Autoscaler vs Karpenter?
13. Helm rollback internally kaise kaam karta hai?
14. Pod-to-pod communication full flow (DNS + iptables)?
15. Kubernetes upgrade strategy — order of operations?

AWS / Cloud (12 Qs)

1. Security Group vs NACL — stateful kaun?
2. VPC mein public vs private subnet ka diff?
3. IAM Role vs IAM User — kab kya?
4. IRSA kya hai aur kyun better than node IAM?
5. S3 bucket public ho gaya — audit + fix?
6. Parameter Store vs Secrets Manager — 4 diff?
7. EKS pod ko S3 access — best practice?
8. Multi-region active-active vs active-passive trade-offs?
9. AWS bill suddenly 3x — investigation flow?
10. RDS Multi-AZ vs Read Replica difference?
11. Spot interruption handle kaise karte ho?
12. ALB vs NLB — kab kya?

IaC + CI/CD (10 Qs)

1. Terraform state corrupt — recovery?
2. Drift detection — production implementation?
3. Pulumi vs Terraform — kab kaunsa?
4. Multi-env (dev/stage/prod) module structure?
5. GitHub Actions OIDC with AWS — full setup?

6. Self-hosted runner vs GitHub-hosted — kab kya?
7. Pipeline mein secrets — securely handle kaise?
8. Blue-green vs Canary — trade-offs?
9. Helm vs Kustomize — kab kya?
10. GitOps (ArgoCD/Flux) ka workflow?

Observability + Debugging (8 Qs)

1. Production app slow — debugging methodology?
2. Prometheus cardinality explosion — kya, kaise avoid?
3. RED method vs USE method?
4. Distributed trace kaise read karte ho?
5. Loki vs ELK trade-offs?
6. Alert fatigue — kaise solve?
7. SLI vs SLO vs SLA — apne project ke examples?
8. OpenTelemetry collector pipeline architecture?

Behavioral (5 Qs)

1. Tumhara biggest production incident — kya hua, kaise solve?
2. Disagreement with team lead — kaise handle?
3. Complex migration story (Vault to SSM perfect)
4. Kyun ye job chod rahe ho? (Diplomatic answer ready)
5. 5 saal mein khud ko kahan dekhte ho?

13. System Design Templates

Universal Framework (15 min response)

- **0-2 min: Clarify** — 'Functional requirements kya hain? Scale (users, requests/sec)? SLA targets? Budget?'
- **2-4 min: Constraints + Trade-offs** — Cost vs perf, consistency vs availability, build vs buy.
- **4-9 min: High-level Architecture** — Boxes + arrows. User → CDN → LB → API → Cache + DB. Components naam kar.
- **9-12 min: Deep Dive** — 1-2 critical components. Articulate trade-offs (chose X over Y because...).
- **12-14 min: Scale + Failure** — 10x grow toh? SPOF kahan? Disaster recovery?
- **14-15 min: Monitoring + Cost** — Key metrics, alerts, rough \$/month estimate.

Template 1: CI/CD for 50 Microservices

Components: Mono-repo (or poly-repo with shared workflows) → GitHub Actions reusable workflows → Build matrix (only changed services) → Test in parallel → Docker build → ECR push → ArgoCD GitOps → EKS deploy. **Trade-offs:** Mono-repo = unified versioning, slow CI. Poly-repo = isolated, sync overhead. **Reusable workflow per language:** Java pipeline, Node pipeline, Python pipeline. **Secrets:** OIDC to AWS, External Secrets in cluster. **Rollback:** ArgoCD app rollback or Helm rollback within 5 min. **Monitoring:** Pipeline duration, success rate, deployment frequency (DORA metrics).

Template 2: Multi-region DR for Stateful Workload

Pattern: Active-passive with warm standby (cost-optimized). **Compute:** EKS in primary (us-east-1) + minimal EKS in DR (us-west-2). **Database:** Aurora Global DB (sub-second cross-region replication, < 1 sec RPO). **Storage:** S3 cross-region replication, async. **DNS:** Route 53 health checks → failover routing. RTO ~5 min. **Failover trigger:** Auto on health check fail OR manual via runbook. **Caveat:** Stateful apps ko handle karna critical — connection draining, in-flight transactions.

Template 3: Observability for 1000-Service Platform

Metrics: Prometheus federation hierarchy. Per-cluster Prom + central Thanos for long-term. **Cardinality control:** Strict label policy, no high-cardinality labels in metrics. **Logs:** Loki for K8s logs (cheap), CloudWatch for AWS service logs. Sampling at high volume. **Traces:** OTel collector → Tempo. 1% sampling at edge, 100% on errors. **Dashboards:** Grafana, dashboards-as-code in Git. **Alerting:** Alertmanager → PagerDuty. SLO-based alerts (burn rate), not threshold spam. **Cost target:** Observability < 5% of infra cost.

14. Behavioral Story Framework (STAR)

Tera Story Bank — 6 Must-Have Stories

1. Vault to SSM Migration (Tech Decision + Migration)

S: Compliance Innovation mein 50+ services HashiCorp Vault use kar rahe the. License cost zyada, ops overhead high.

T: AWS-native solution pe migrate karna without downtime aur audit trail maintain karke.

A: Designed parallel-sync approach using ESO. Set up SSM Parameter Store, configured ESO with SecretStore. Created CRD-based ExternalSecret manifests. Tested in dev, staged in test, gradual prod cutover. Maintained Vault running for 2 weeks rollback window.

R: 50+ services migrated, zero downtime, [X% cost reduction], audit trail via CloudTrail, simplified ops.

2. Docker Compose to Helm Migration (Architecture)

S: Indicios stack Docker Compose pe tha — local dev fine, but no orchestration in prod, manual scaling.

T: Migrate to Kubernetes with Helm charts for multi-env consistency.

A: Created Helm chart with parametrized values for dev/test/prod. Used Pulumi to provision EKS + chart deployment. Implemented health checks, resource limits, horizontal autoscaling. Added Grafana dashboards for observability.

R: Multi-env parity, automated scaling, [Y% deployment time reduction], end-to-end observability.

3. Production Incident (Failure Story)

S: [Choose ek real incident — kuch chhota lekin learning wala. Example: secret rotation broke a service, DNS misconfig, wrong terraform apply.]

T: Restore service quickly + RCA.

A: Triage steps batao — logs check, alerts, rollback, communication. What went wrong specifically.

R: [Resolution time], post-mortem learnings, process changes (more monitoring, better runbook, automation).

4. Cross-team Disagreement (Conflict)

S: [Example: Dev team chahti thi direct EKS access, security team push back. Ya: monolith vs microservices debate.]

T: Find solution balancing both needs.

A: Listened to both, proposed middle ground (e.g., GitOps with PR-based access for devs + RBAC for security). Documented trade-offs, demo'd POC.

R: Both teams agreed, [outcome metric].

5. Database DevOps for Verizon (Leadership)

S: Victra (Verizon) project mein DB deployments manual the — error-prone, slow.

T: Lead end-to-end Database DevOps with SSDT.

A: Designed Azure DevOps pipelines for SSDT projects, automated build → test → deploy across envs. Created branching strategy, rollback procedure, set up alerting.

R: [X% faster deployments, Y% fewer prod issues, audit-compliant].

6. Why Leaving (Diplomatic + True)

S: Currently at TokenTide doing infra work.

T: Looking for [larger scale / more diverse tech / stronger engineering culture / specific learning].

A: Researched companies, applying selectively.

R: Looking for next role where I can [specific contribution + grow].

15. Final Cheatsheet (Day-of-Interview)

Interview ke subah 30 min mein revise karega ye section. Sab kuch nahi padhna — sirf ye.

5 Things to Remember

- **STAR format:** Situation, Task, Action, Result. Numbers add karna mandatory.
- **Honest about gaps:** 'Recently kaam nahi kiya, but concept ye hai aur quickly pick up kar sakta hu.' Bluffing pakdi jaati hai.
- **Loud think:** Code/design problem mein soch loud bolke share karo — interviewer ko process dikhayo.
- **Ask questions:** Last mein 3-5 questions interviewer se. Silence = bad signal.
- **Calm:** Pause leke jawab do. 'Achha question, ek second sochta hu' — totally OK.

Top Concepts to Have on Tip

- **K8s:** Pod lifecycle, probes (liveness restart, readiness remove from svc), QoS, services, RBAC.
- **AWS:** SG vs NACL, IRSA, S3 access patterns, VPC architecture, Karpenter advantage.
- **Networking:** TCP handshake, HTTP status codes, DNS records, TLS basics, L4 vs L7 LB.
- **IaC:** TF state management, drift detection, Pulumi vs TF when, module versioning.
- **CI/CD:** OIDC modern, reusable workflows, deployment strategies, branching trunk-based.
- **Observability:** RED vs USE, cardinality, OTel, SLI/SLO/SLA, debugging methodology.

Final Pep Talk

Bhai, ek aakhri baat.

Tu khud ko 'weak' bol raha tha. Lekin sach ye hai — tune 5 saal mein 3 cloud platforms pe production deploy kiya, CKA pass kiya, Vault se SSM migration kiya, Docker Compose se Helm migration kiya, Database DevOps lead kiya Verizon ke project pe. Ye chhote kaam nahi hain. Ye real engineering hai.

Tu rusty hai, weak nahi. Difference samajh — rust 15 din mein hat jaati hai, weakness 5 saal mein.

Daily 3 ghante focused — 10 baje shuru, phone band, distractions zero. 15 din mein tu naya banda hoga.

Family ke liye, biwi ke liye, apne future ghar ke liye — ye 15 din kaam kar. Tu kar lega. Bharosa rakh.

— **All the best, Sameer. Tu Khandwa se Bangalore-Pune jaisi har company crack kar sakta hai.**